Random Forests

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026

THOMPSON RIVERS UNIVERSITY

Topics

Recap

- Last time we covered **Bootstrap Aggregation (Bagging)**:
  - Bootstrap sampling review.
  - Ensemble methods.
  - Bagged trees.
  - Variable importance.

Introduction

- Tree methods are easy to construct and relatively easy to interpret.
- In the presence of high variability, they can struggle to generalize to new data.
  - **We applied bagging to help with this.**
- Tree correlation prevents bagging from further reducing the variance of the base learner.
  - **We can use random forests to help with this.**

Random Forests

Introduction to Random Forests

- **Random forests** are a modification of bagged trees that build a large collection of de-correlated trees to further improve predictive performance.

- Very popular machine learning tool.
  - *Out-of-the-box* performance is usually very good and tuning variability is minimal.

- First published by Leo Breiman in 2001.

Random Forests

- Random forests help to reduce tree correlation by adding more *randomness* into the tree growing process.

- While growing a decision tree during the bagging process, random forests perform **split-variable randomization** at each split.
  - The search for the split variable is limited to a random subset of m of the original p variables.

- Typical default values for the tuning parameter m:
  - $m = \frac{p}{3}$ (regression).
  - $m = \sqrt{p}$ (classification).
  - $m = p$ (bagging).

Procedure

- Obtain training data.
- Select the number of trees to build.
- For each tree to be built, do the following:
  1. Generate a bootstrap sample of the original training data.
  2. Grow a regression/classification tree to the bootstrapped data
  3. At each split do:
     1. Select m variables at random from all p variables.
     2. Pick the best variable/split-point among the m.
     3. Make the binary partition.
  4. Grow the tree completely (do not prune).
- Return the averaged prediction.

randomForest Package

- We can use the randomForest R package.
  - In Python.

- Usage:

```
RF_model <- randomForest(Y~ X1 + ...  + XP,
data = Wages,
mtry = m,
importance = TRUE,
ntree = 500)

plot(RF_model) To plot OOB sample error.
```

Out-of-Bag Validation

- In the process of bootstrapping, not all of the observations will be in the sample.
  - On average approximately 63.21%.
- The **out-of-bag (OOB) sample** are the observations not included in the bootstrapped sample used to train the individual model.
- The OOB sample can be used to test predictive performance.
  - The results usually compare well compared to k-fold CV (sufficient sample size $n \geq 1000$).

Example 1

- Import the *diamonds* dataset from the ggplot2 package and do the following:

  1. Set your seed and perform a 80/20 train-test split.
  2. Construct a random forest of 500 trees for the `price` of the diamonds using $m = \text{floor}(p/3)$.
  3. Plot the OOB Error.

- What do you notice?

Out-of-Box Performance

- Generally, random forests perform fairly well without tuning.
  - *Initial model performed fairly well.*
- There are some hyperparameters that we can tune to improve things.
- Note: *If you want example code to check the number of trees to see how the model generalizes to a validation set see this document.*

Hyperparameters

Hyperparameters

- The following parameters may be tuned to improve accuracy of random forests:
  1. The number of trees in the forest.
  2. The number of variables to consider at any given split: m.
  3. The complexity of each tree.
  4. The sampling scheme.
  5. The splitting rule to use during tree construction.

- *In order of impact on prediction accuracy.*

(1) Number of Trees

- *Not technically a hyperparameter.*
- More trees provide more robust and stable error estimates and variable importance measures.
  - **Linear relationship with the number of trees and computational demands.**
- As you adjust m and node sizes, you may need to adjust the number of trees.
- Typically, start with $p \times 10$ for the number of trees.

(2) The Number of Variables (m)

- Helps to balance low tree correlation with reasonable predictive strength.
- Typical default values for the tuning parameter m:
  - $m = \frac{p}{3}$ (regression).
  - $m = \sqrt{p}$ (classification).
  - $m = p$ (bagging).
- Higher values of m tend to perform better when there are fewer relevant variables.

Example 2

- Using your training set of *diamonds* from Example 1, do the following:
  1.
  2. Construct the following random forests:
     1. 500 trees, $m = 3$.
     2. 500 trees, $m = 5$.
     3. 900 trees, $m = 3$.
     4. 900 trees, $m = 5$.
  3. Plot the OOB Errors.

- What do you notice?

(3) Tree Complexity

- When we tuned single trees, we could adjust node size and max depth (early stopping).
- Node size is the most commonly tuned complexity hyperparameter.
- Defaults:
  - 1 for classification.
  - 5 for regression.
- If your model performs better with a larger m, increasing node size may help.
  - *Noisy data.*

(4) Sampling Scheme

- Default sampling scheme for random forests is bootstrapping where 100% of the observations are sampled with replacement.
- Can adjust both the sample size and whether to sample with or without replacement.
- Decreasing the sample size leads to more diverse trees and thereby lower between-tree correlation.
- When you have many categorical features with a varying number of levels, sampling with replacement can lead to biased variable split selection.

(5) Splitting Rule

- The default splitting rule during random forests tree building consists of selecting, out of all splits of the (m) candidate variables, the split that minimizes:
  - Gini impurity (Classification).
  - SSE (Regression).

- *This approach has been shown to favour continuous variables.*

- To increase computational efficiency, splitting rules can be randomized where only a random subset of possible splitting values is considered.

Grid Search Illustrative Example

- We can use the *ranger* package in R.
- The provided code shows an example of a Cartesian grid search of hyperparameters using the *Wages.csv* dataset.
- *These grid searches can be computationally demanding.*

Example 3

1. Using your training set of *diamonds* from Example 1, adjust the code from Grid Search Illustrative Example and find the *best* random forest.
2. Use your identified *best* solution to construct a random forest.
3. What do you notice?

Tuning Comments

- Cartesian searches can be very computationally demanding.
- You can use the *h2o* package to perform a random search with early stopping capabilities.
  - An example can be found in textbook (4).
- Typically, the first two tuning parameters that we covered will have the largest impact on accuracy.

Example 4

- Use your model from Example 1, the best model from Example 2, and the model from Example 3 to compare the prediction accuracies on the testing dataset.

- What do you notice?

Variable Importance

Variable Importance

- We can estimate variable importance similar to how we have in previous sections.

- We can also look at *permutation-based* importance measure.
  - *Shuffles the variables as they are put into the trees.*
  - Variables that when moved, result in a decrease in accuracy are deemed important.

- Using the *ranger* package we can add the following arguments:
  - `importance = "impurity"` (classic approach).
  - `importance = "permutation"` (permutation-based approach).

  - `vip::vip(model)`

Example 5

- Using the best combination of hyperparameters from Example 3:
  1. Adjust the provided code to construct two random random forests.
  2. Identify the most important variables.

- What do you notice?

Final Thoughts

- Random forests improves upon bagging by reducing the tree correlation.
- Generally, they perform well *out-of-the-box* and do not require much tuning.
  - Adjusting the number of trees and m should provide the most improvements.
- *Very commonly used machine-learning algorithm.*

Next Lecture

- We will discuss **Dimension Reduction**.
- Procedure.
- PCA.

Exercise 1

- Import the wine dataset from the *HDclassif* package.
- Create a 75-25 train/test split.
- Construct an out-of-the box random forest for your training dataset.
- Perform a Cartesian search for the *best* combination of hyperparameters.
- Construct a random forest with the *best* tuning parameters.
- Finally, compare the classification accuracy of all of your trees on the testing set.
- What do you notice?

References & Resources

1. Bakker, J. D. (2026) Applied Multivariate Statistics in R
2. Peng R. D. (2020) *Exploratory Data Analysis with R*
3. Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction.* Chapman and Hall/CRC
4. Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R.* Taylor & Francis

- bagging()
- rpart()
- rpart.plot()
- vip()
- randomForest
- ranger